Data Structures and Algorithm Analysis
Quiz 3 Practice

10-607

---

1. Given a binary search tree with integer valued nodes, provide an algorithm to recover a sorted list of elements. In your description of your algorithm, prove both correctness and time-complexity with the invariants of the BST data structure.

Ans: Algorithm (in-order traversal). Let each node have fields `key`, `left`, and `right`. We maintain a global list `L` initially empty.

```
def inorder(node):
    if node is None:
        return
    inorder(node.left)
    L.append(node.key)
    inorder(node.right)

# call:
L = []
inorder(root)
# now L contains all keys in sorted order
```

BST invariant. For every node x:

$$\text{all keys in left subtree of } x < \text{key}(x) < \text{all keys in right subtree of } x.$$

Correctness. We prove by induction on the number of nodes n in a (sub)tree T that calling `inorder(T)` appends the keys of T to `L` in strictly increasing order.

Base case ($n = 0$). If T is empty (`node is None`), the function returns immediately and appends nothing. The resulting list is (trivially) sorted.

Inductive step. Suppose the claim holds for all trees with fewer than n nodes. Let T be a tree with root r and subtrees T_ℓ (left) and T_r (right). By the inductive hypothesis:

- The recursive call `inorder(r.left)` appends all keys in T_ℓ to `L` in increasing order.

- The recursive call `inorder(r.right)` appends all keys in T_r to `L` in increasing order.

By the BST invariant, every key in T_ℓ is $<$ key(r) and every key in T_r is $>$ key(r). The algorithm appends:

$$(\text{sorted keys of } T_\ell),\ \text{key}(r),\ (\text{sorted keys of } T_r),$$

and this concatenation is still strictly increasing. Thus the keys of T are output in sorted order.

By induction, the entire tree rooted at `root` yields `L` sorted in nondecreasing order.

Time complexity. Each node is visited once, and on each visit we perform $O(1)$ work (constant number of pointer checks and at most one append). There are n nodes, so the total time is $O(n)$. The recursion depth is $O(h)$ where h is the height of the tree (at most n in the worst case).

2. Provide the recurrences and runtime complexity (big-O) of the following algorithms:

   (a) Algorithm A solves a problem of size n by dividing it into 4 subproblems, each of size n/3, solving each subproblem, and then combining the solutions in linear time.

   (b) Algorithm B solves a problem of size n by dividing it into nine subproblems, each of size n/2, solving each subproblem, and then combining the solutions in quadratic time.

Ans:

We first write down the recurrences, then solve them by a change of variables and unrolling.

(a) Algorithm A:

There are 4 subproblems of size $n/3$. Combining their answers costs cn for some constant $c > 0$.

So the recurrence is

$$T(n) = 4\,T\Big(\frac{n}{3}\Big) + cn.$$

To solve it, assume for simplicity that n is a power of 3, say $n = 3^k$. Define

$$S(k) := T(3^k).$$

Then

$$S(k) = T(3^k) = 4T(3^{k-1}) + c \cdot 3^k = 4S(k-1) + c3^k.$$

Now we unroll this recurrence:

$$\begin{aligned} S(k) &= 4S(k-1) + c3^k \\ &= 4\big(4S(k-2) + c3^{k-1}\big) + c3^k \\ &= 4^2 S(k-2) + c4 \cdot 3^{k-1} + c3^k \\ &\vdots \\ &= 4^k S(0) + c\sum_{i=1}^{k} 4^{k-i}3^i. \end{aligned}$$

Factor out 4^k from the sum:

$$\sum_{i=1}^{k} 4^{k-i}3^i = 4^k \sum_{i=1}^{k} \left(\frac{3}{4}\right)^i.$$

So

$$S(k) = 4^k S(0) + c4^k \sum_{i=1}^{k} \left(\frac{3}{4}\right)^i = 4^k \left(S(0) + c\sum_{i=1}^{k} \left(\frac{3}{4}\right)^i\right).$$

The sum $\sum_{i=1}^{k}(3/4)^i$ is a geometric series with ratio < 1, so it is bounded above and below by positive constants independent of k. Therefore

$$S(k) = \Theta(4^k).$$

Now recall $n = 3^k$, so $k = \log_3 n$ and

$$4^k = 4^{\log_3 n} = n^{\log_3 4}.$$

Thus

$$T(n) = S(\log_3 n) = \Theta\big(n^{\log_3 4}\big),$$

so in particular $T(n) = O\big(n^{\log_3 4}\big)$.

(For general n that are not exact powers of 3, we can round n up or down to the nearest power of 3; this only changes $T(n)$ by a constant factor and does not affect the big-O.)

(b) Algorithm B:

There are 9 subproblems of size $n/2$. Combining their answers costs cn^2 for some constant $c > 0$.

So the recurrence is

$$T(n) = 9\,T\Big(\frac{n}{2}\Big) + cn^2.$$

Again, assume n is a power of 2, say $n = 2^k$, and set

$$S(k) := T(2^k).$$

Then

$$S(k) = 9S(k-1) + c2^{2k} = 9S(k-1) + c4^k.$$

Unroll as before:

$$\begin{aligned} S(k) &= 9S(k-1) + c4^k \\ &= 9\big(9S(k-2) + c4^{k-1}\big) + c4^k \\ &= 9^2 S(k-2) + c9 \cdot 4^{k-1} + c4^k \\ &\;\;\vdots \\ &= 9^k S(0) + c\sum_{i=1}^{k} 9^{k-i}4^i. \end{aligned}$$

Factor out 9^k:

$$\sum_{i=1}^{k} 9^{k-i}4^i = 9^k \sum_{i=1}^{k} \left(\frac{4}{9}\right)^i.$$

So

$$S(k) = 9^k \left(S(0) + c\sum_{i=1}^{k} \left(\frac{4}{9}\right)^i \right).$$

Again, $\sum_{i=1}^{k} (4/9)^i$ is a bounded geometric series, so

$$S(k) = \Theta(9^k).$$

With $n = 2^k$, we get

$$9^k = 9^{\log_2 n} = n^{\log_2 9},$$

hence

$$T(n) = S(\log_2 n) = \Theta\big(n^{\log_2 9}\big),$$

and in particular $T(n) = O\big(n^{\log_2 9}\big)$.

So the final answers are:

$$\boxed{\begin{aligned} &\text{(a)} \quad T(n) = 4T\Big(\frac{n}{3}\Big) + cn \quad \Rightarrow \quad T(n) = \Theta\big(n^{\log_3 4}\big); \\ &\text{(b)} \quad T(n) = 9T\Big(\frac{n}{2}\Big) + cn^2 \quad \Rightarrow \quad T(n) = \Theta\big(n^{\log_2 9}\big). \end{aligned}}$$

3. We are given a linked list list of distinct integers.

   a) Suppose that the linked list is sorted. Provide an O(1) time algorithm to construct a binary search tree from this linked list. What is the lookup time of any element in the resultant search tree?

   b) Suppose that the list is not sorted. What is the time complexity of constructing a binary search tree from this list?

   c) We say a tree is *balanced* when the left and right subtree height/depth of each node differs by at most 1. Discuss how we can implement a binary tree interface that has this invariant built-in.

Ans: Let the list contain distinct keys in increasing order.

(a) Sorted list, $O(1)$ construction. Conceptually, we treat the linked list as a *degenerate* BST:

- The head of the list becomes the root of the BST.
- For each node, its "next" pointer is used as its *right* child.
- Every node's *left* child is considered `null`.

We do not need to touch or rearrange any nodes; we just store a pointer to the head and agree to interpret the `next` pointers as right-child pointers. This is $O(1)$ time.

Why is this a BST? Because the list is sorted increasingly, as we follow the right-child pointers the keys strictly increase. Each node has no left child, so the BST invariant (all keys in left subtree $<$ key $<$ all keys in right subtree) holds.

Lookup time. The tree is a chain of n nodes; its height is $n - 1$. A standard BST lookup follows a path from the root down the tree. In the worst case we may traverse all n nodes, so lookup is $O(n)$ time.

(b) Unsorted list. A natural approach is to start with an empty BST and *insert* each list element one by one:

- Each insertion takes $O(h)$ time where h is the current height.
- In the worst case, the BST can degenerate into a chain of length $O(n)$, giving $O(n)$ per insertion and total $O(n^2)$ time.
- If the keys arrive in a "random" order and we do not rebalance, the expected height is $O(\log n)$, so total expected time is $O(n \log n)$.

So:

worst case: $O(n^2)$, typical/average case (no rebalancing): $O(n \log n)$.

(c) Keeping the tree balanced. To build a binary tree interface whose trees remain balanced, we use a *self-balancing BST* such as an AVL tree or a red–black tree:

- Each node stores extra information (e.g. height or a color).
- After every insertion or deletion, we check the local balance condition at affected nodes.
- If the heights of left and right subtrees differ by more than 1, we perform one or two *rotations* (pointer rearrangements) to restore balance.

These rebalancing operations take $O(1)$ time per level, and they only act along the search path, so overall insert and delete operations run in $O(\log n)$ time and the tree height stays $O(\log n)$. The public interface (search/insert/delete) looks like a normal BST, but the implementation guarantees the "balanced" invariant internally.

4. Provide an algorithm to reverse a queue in $O(N)$ time complexity and $O(1)$ space complexity. Suppose we are on the client side, so you may not assume anything about the underlying implementation of the queue (e.g arrays or linked lists or doubly linked lists). You only have access to the following functions: deq(q), enq(q, elem), and size(q).

   Follow up: Now, assume we are on the library side of the implementation. We want to create a function called reverse(q) that reverses a given queue in $O(N)$ time complexity and $O(1)$ space complexity. We know that our queue is implemented with a singly-linked list. Provide your algorithm.

Ans: Client side (queue as an abstract ADT).

We only have `deq`, `enq`, and `size`. A simple recursive solution is:

```
def reverse(q):
    if size(q) <= 1:
        return
    x = deq(q)          # remove front element
    reverse(q)          # reverse the remaining queue
    enq(q, x)           # put x at the back
```

Correctness (by induction on $n =$ `size`(q)).

- Base cases $n = 0, 1$: the queue is already its own reverse; the algorithm returns immediately.
- Inductive step: assume the procedure correctly reverses any queue of size $< n$. Let the current queue be

  $$[x, a_2, a_3, \ldots, a_n].$$

  After `deq`, we have x stored in a variable, and the queue is $[a_2, \ldots, a_n]$. By the induction hypothesis, `reverse(q)` reverses it to $[a_n, \ldots, a_2]$. Finally `enq(q,x)` yields $[a_n, \ldots, a_2, x]$, which is exactly the reverse of the original.

Time complexity. Each element is dequeued once and enqueued once. The recursion makes n calls, each doing $O(1)$ work besides recursive calls. Total time is $O(n)$.

Space complexity. We use $O(1)$ extra variables, but the recursion uses $O(n)$ call-stack frames. If the call stack is counted, the auxiliary space is $O(n)$; if "space complexity" is defined to ignore recursion stack, then it is $O(1)$ in that sense. (Good exam discussion point.)

Library side (we know it's a singly-linked list).

Now we can manipulate the underlying linked list directly. Suppose the queue struct stores pointers `head` and `tail` into a singly linked list:

```
typedef struct Node {
    int value;
    struct Node* next;
} Node;

typedef struct Queue {
```

```
    Node* head;  // front of queue
    Node* tail;  // back of queue
    int size;
} Queue;

void reverse(Queue* q) {
    Node* prev = NULL;
    Node* curr = q->head;
    Node* next = NULL;
    Node* old_head = q->head;

    while (curr != NULL) {
        next = curr->next;    // remember next node
        curr->next = prev;    // reverse the pointer
        prev = curr;          // advance prev
        curr = next;          // advance curr
    }

    // After the loop: prev points to old tail (new head)
    q->head = prev;
    q->tail = old_head;
}
```

Correctness. The loop is the standard in-place reversal of a singly-linked list:

- At each step, the pointer from `curr` is flipped to point backward to `prev`.
- The invariant is: all nodes before `curr` now form a reversed prefix list ending at `prev`, and the sublist starting at `curr` is the unreversed suffix.
- At termination, `curr` is `NULL` and `prev` is the old tail, which is now the first node of the reversed list.

We then set `head` to the new front (`prev`) and `tail` to the old head. All edges are reversed and the order of elements in the list is reversed, which is exactly what we want for the queue.

Complexity. We traverse each node once and perform $O(1)$ work per node, so the time is $O(n)$. We use a constant number of pointers, so extra space is $O(1)$.

5. Suppose we have a undirected graph $G = (V, E)$. Let $s, t \in V$ be two arbitrary vertices. We wish to discover whether an $s-t$ path exists in G. Provide an algorithm

that checks for an $s - t$ path in G. Your algorithm should run in $O(V + E)$ time.

Follow-up question: now suppose we want to see if there exists a $s - t$ path that uses less than k edges. How can we see if there is such a path?

Ans:

Part 1: Does any s–t path exist?

We can use either DFS or BFS. Here is BFS with an adjacency list:

```
from collections import deque

def path_exists(G, s, t):
    # G: adjacency list, e.g. dict v -> list of neighbors
    visited = {v: False for v in G}
    q = deque()

    visited[s] = True
    q.append(s)

    while q:
        u = q.popleft()
        if u == t:
            return True
        for v in G[u]:
            if not visited[v]:
                visited[v] = True
                q.append(v)

    return False
```

Correctness.

- If there is an s–t path, BFS explores all vertices reachable from s by repeatedly enqueuing neighbors of visited vertices. Since t is reachable, eventually it is dequeued and we return `True`.
- If there is no s–t path, then t is not reachable from s. BFS only visits vertices in the connected component of s, so t is never enqueued and the function eventually returns `False`.

Time complexity. Each vertex is enqueued and dequeued at most once, and each edge is considered at most twice (once from each endpoint) in an undirected graph. Thus the running time is $O(|V| + |E|)$.

Follow-up: Path with fewer than k edges.

We again run BFS from s, but now we record the *distance in edges* from s:

```
from collections import deque
INF = 10**9

def path_with_less_than_k_edges(G, s, t, k):
    dist = {v: INF for v in G}
    q = deque()

    dist[s] = 0
    q.append(s)

    while q:
        u = q.popleft()
        for v in G[u]:
            if dist[v] == INF:
                dist[v] = dist[u] + 1
                q.append(v)

    # If t is reachable and its shortest path length is < k:
    if dist[t] < k:
        return True
    else:
        return False
```

Correctness. BFS on an unweighted graph has the key property that *distances from s in the BFS tree equal the minimum number of edges in any s–vertex path.* So:

- If there exists an s–t path with fewer than k edges, then the shortest s–t path has length $d < k$, and BFS will set `dist[t] = d`, so the algorithm returns `True`.
- If no s–t path uses fewer than k edges, then either t is unreachable (so `dist[t]` remains `INF` and we return `False`), or the shortest path length is at least k, in which case `dist[t] >= k` and we again return `False`.

Time complexity. This is just a normal BFS with an extra `dist` array. As before, each vertex and edge is processed $O(1)$ times, so the running time is $O(|V| + |E|)$.

6. Consider the following undirected weighted graph $G = (V, E)$ with vertices

$$V = \{A, B, C, D, E, F\}$$

and edges (with weights in parentheses):

$$E = \{\, AB(4),\ AC(2),\ BC(1),\ BD(5),\ CD(8),\\ CE(10),\ DE(2),\ DF(6),\ EF(3)\,\}.$$

(a) Use **Kruskal's algorithm** to compute a minimum spanning tree (MST) of G. Show the order in which edges are considered and which edges are added or skipped. State the final MST and its total weight.

(b) Use **Dijkstra's algorithm** to compute the shortest-path distances from source vertex A to all other vertices. Show the distances after each iteration when a new vertex is finalized. Draw (or list) the final shortest-path tree rooted at A.

Ans: First sort edges by weight:

$$BC(1),\ AC(2),\ DE(2),\ EF(3),\ AB(4),\ BD(5),\ DF(6),\ CD(8),\ CE(10).$$

(a) Kruskal's algorithm (MST).

We scan edges in this order, adding an edge if it does not create a cycle.

- Take $BC(1)$: add (no cycle yet).
- Take $AC(2)$: add (connects A to $\{B, C\}$).
- Take $DE(2)$: add (new component $\{D, E\}$).
- Take $EF(3)$: add (component $\{D, E, F\}$).
- Take $AB(4)$: skip (would create a cycle in $\{A, B, C\}$).
- Take $BD(5)$: add (merges $\{A, B, C\}$ and $\{D, E, F\}$).
- Remaining edges: skip (graph already connected).

MST edges:

$$\{\, BC(1),\ AC(2),\ DE(2),\ EF(3),\ BD(5)\,\},$$

with total weight

$$1 + 2 + 2 + 3 + 5 = 13.$$

Answer: one valid MST is $T = \{BC, AC, DE, EF, BD\}$ with $w(T) = 13$.

(b) Dijkstra's algorithm from A.

We keep distances $d[\cdot]$ from A and mark vertices as "finalized" when their distance is smallest among all unfinalized vertices.

Initialization:

$$d(A) = 0, \quad d(B) = d(C) = d(D) = d(E) = d(F) = \infty.$$

- Finalize A (distance 0). Relax neighbors:

$$d(B) = 4, \quad d(C) = 2.$$

- Next smallest unfinalized is C (distance 2). Relax neighbors:

$$d(B) \leftarrow \min(4,\, 2 + 1) = 3, \quad d(D) = 2 + 8 = 10, \quad d(E) = 2 + 10 = 12.$$

- Next smallest is B (distance 3). Relax neighbors:

$$d(D) \leftarrow \min(10,\, 3 + 5) = 8.$$

- Next smallest is D (distance 8). Relax neighbors:

$$d(E) \leftarrow \min(12,\, 8 + 2) = 10, \quad d(F) = 8 + 6 = 14.$$

- Next smallest is E (distance 10). Relax neighbor:

$$d(F) \leftarrow \min(14,\, 10 + 3) = 13.$$

- Finally, F (distance 13) is finalized; no further changes.

Final shortest-path distances from A:

$$d(A) = 0, \quad d(C) = 2, \quad d(B) = 3, \quad d(D) = 8, \quad d(E) = 10, \quad d(F) = 13.$$

Using predecessors from the relaxations, one shortest-path tree rooted at A is:

$$A \to C, \quad C \to B, \quad B \to D, \quad D \to E, \quad E \to F,$$

i.e. tree edges

$$\{\, A{-}C,\ C{-}B,\ B{-}D,\ D{-}E,\ E{-}F\,\}.$$